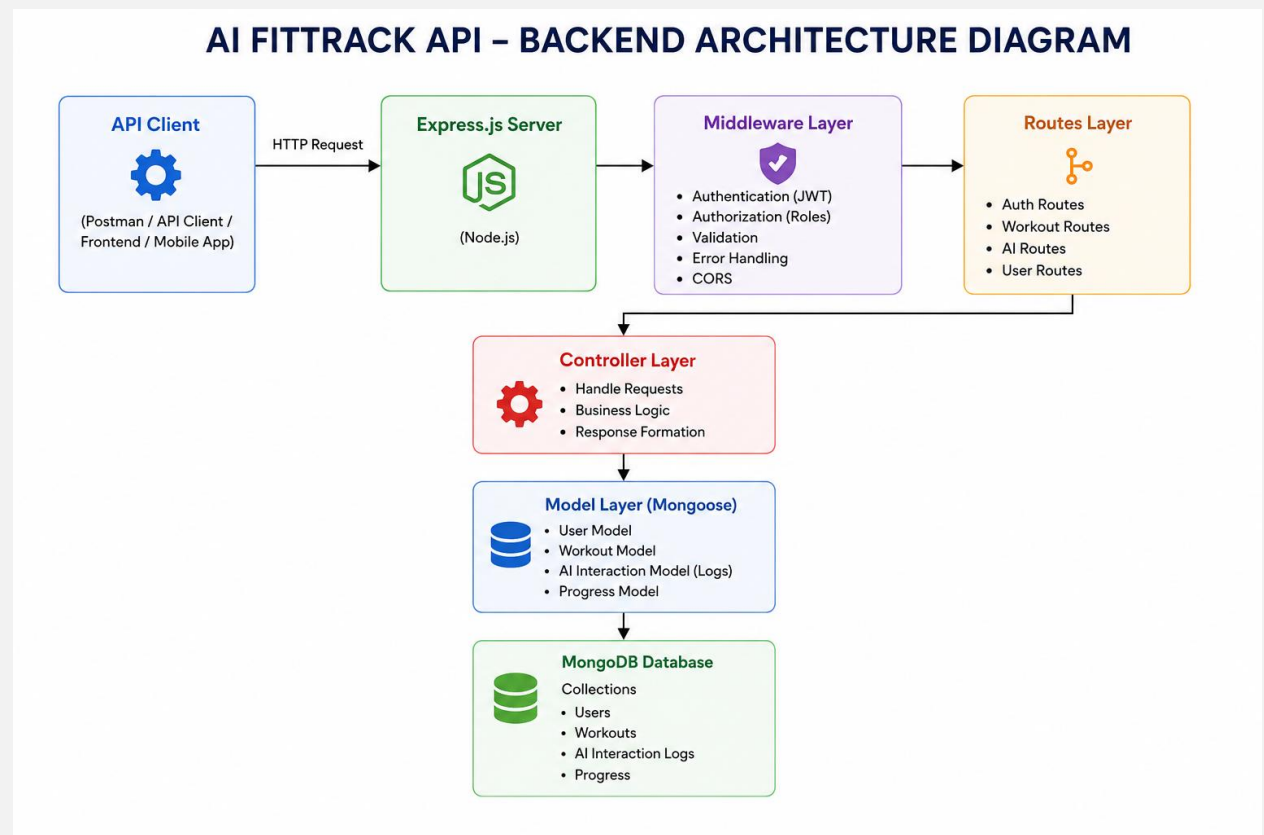




Project Design Phase

Epic 1: Project Architecture



The FitTrack AI follows a layered architecture based on the MVC (Model–View–Controller) design pattern. The application separates routing, business logic, database interaction, middleware, and AI services into independent modules, making the project easier to maintain and scale.

When a client sends an HTTP request, the Express server receives it and forwards it to the corresponding route. Protected routes first pass through the authentication middleware, which validates the JWT token before granting access. Controllers process validated requests and invoke business logic or AI services as required. Database interactions are handled through Mongoose models, while Google Gemini AI is accessed through a dedicated service layer. Finally, the controller returns a structured JSON response to the client.

This layered approach improves modularity, security, and maintainability while ensuring that each component has a single responsibility.

Overall Architecture Components

- Client Application (React / Mobile App / Postman)
- Express Server
- API Routes
- Authentication Middleware

- Controllers
- Services
- Google Gemini AI Service
- MongoDB Database
- Mongoose Models

Each component communicates through well-defined interfaces, ensuring loose coupling and easier future enhancements.

Components Definition

The FitTrack AI follows a modular backend architecture in which every component performs a dedicated responsibility. The separation of concerns ensures maintainability, scalability, and efficient execution of requests. Each request passes through multiple layers before reaching the database or AI service, ensuring secure and reliable processing.

1. Client Layer (Frontend/API Consumer)

The Client Layer is the entry point of the application. It can be a React application, mobile application, or API testing tool such as Postman. The client sends HTTP requests to the backend server and receives JSON responses.

Responsibilities

- Sends API requests.
- Displays workout information.
- Stores JWT token after login.
- Invokes AI recommendation APIs.
- Displays success and error responses.

APIs Accessed

- Authentication APIs
- Workout APIs
- AI Recommendation APIs
- AI Fitness Insight APIs

2. Express Server Gateway

The Express server acts as the central gateway of the application. It listens for incoming requests, configures middleware, parses request bodies, enables CORS, and routes requests to the appropriate controllers.

Responsibilities

- Starts HTTP server
- Parses JSON requests
- Enables Cross-Origin Resource Sharing (CORS)
- Registers application routes
- Connects middleware
- Handles centralized errors

Example Responsibilities

- Receive HTTP Request
- Validate Request Format
- Forward Request to Routes
- Return JSON Response

3. Authentication Middleware

Authentication middleware protects all secured APIs by validating the JSON Web Token (JWT). Every protected request must include a valid Authorization header.

Responsibilities

- Read Authorization Header
- Extract JWT Token
- Verify JWT Signature
- Decode User Information
- Attach User Information to Request
- Reject Unauthorized Requests

Benefits

- Prevents unauthorized access
- Secures workout information
- Allows only authenticated users to access protected APIs
- Eliminates repeated authentication logic inside controllers

4. Route Layer

Routes define the application's API endpoints and map incoming HTTP requests to controller methods. Each route represents a specific functionality of the system.

Authentication Routes

- Register User
- Login User
- User Profile

Workout Routes

- Add Workout
- Get All Workouts
- Search Workouts
- Get Workout by ID
- Update Workout
- Delete Workout

AI Routes

- Workout Recommendation
- Fitness Insights

5. Controller Layer

Controllers contain the application's business logic. They receive requests from routes, validate input data, invoke services or database operations, and generate standardized JSON responses.

Authentication Controller

Responsible for:

- User Registration
- User Login
- User Profile Retrieval

Workout Controller

Responsible for:

- Creating Workout Records
- Retrieving Workout Records
- Updating Workouts
- Deleting Workouts

- Searching Workouts

AI Controller

Responsible for:

- Sending prompts to Google Gemini
- Receiving AI-generated recommendations
- Returning AI responses to clients

6. Service Layer

The Service Layer contains reusable business logic that can be shared among multiple controllers.

Gemini AI Service

Responsible for:

- Preparing prompts
- Sending prompts to Google Gemini
- Receiving AI-generated responses
- Returning formatted recommendations

JWT Service

Responsible for:

- Creating JWT Tokens
- Configuring Token Expiry
- Verifying User Identity

Password Service

Responsible for:

- Hashing Passwords
- Comparing Passwords During Login

7. Database Layer

MongoDB stores all persistent application data. Mongoose acts as an Object Data Modeling (ODM) library that simplifies communication between Node.js and MongoDB.

Collections

- Users Collection
- Workouts Collection

Advantages

- Flexible Schema
- High Performance
- Easy Data Retrieval
- Scalable Architecture